

Dataset paths

1.1

ParaVision dataset paths have the following form:

<DataPath>/<name>/<expno>/pdata/<procno>

Table 1.1 describes the components of a dataset path:

Component	Description
<DataPath>	This is an arbitrary path in the directory tree that is used as root for <i>ParaVision</i> datasets.
<name>	This directory contains all information about a Study . The name is created by <i>ParaVision</i> . The maximum length of the directory name is 64 characters.
<expno>	The EXPNO directory contains all information to describe one experiment. The directory name is the experiment number.
<procno>	The PROCNO directory contains all information about a reconstructed or derived image series of the experiment. Several PROCNOs may exist which contain different derived image series (e. g. different reconstructions, parameter images, etc.). The directory name is the PROCNO (processed images) number.

Table 1.1: Components of a dataset path

The default <DataPath> setting is <PvInstDir>/<USER>.

These directories contain data files (image data, raw data) and parameter files.

The following tables show an overview of the most important files in the different component directories. They describe only files that may be created by the current *ParaVision* version. Files created by *TOPSPIN* are not described here. Please refer to the *TOPSPIN* manual. All files in the dataset directories may change in future versions.

The **Study** directories may contain the following files:

Filename	Type	Paramter Group	Description
subject	Parameter	SUBJECT	Contains all patient/subject and study parameters.
AdjStatePerStudy	Parameter	AdjStatePer-Study	Contains information about the last executed study adjustments (date, successful, not successful). It does not contain the adjusted values for this Study .
ResultState	Parameter	AdjResult-Group"	Contains information about the results of the last executed studies. It does not contain the adjusted values for this Study but contains references to the results.
AdjResult	Directory		Contains subdirectories for every adjustment result. The result parameters are stored in the file <code>result.jcamp</code> in these subdirectories.

Table 1.2: Content of Study directories

ScanProgram.scanProgram	XML		Information from the database about the study. Is only valid if study is completed.
-------------------------	-----	--	---

Table 1.2: Content of Study directories

The **EXPNO** directories may contain the following files:

Filename	Type	Parameter Group	Description
acqp	Parameter	ACQP	Describes the actual acquisition for this experiment.
AdjStatePerScan	Parameter	AdjStatePer-Scan	Contains information about the last PVM per Scan adjustments (date, successful, not successful). It does not contain the adjusted values for this Scan.
configscan	Parameter	CONFIG_SCAN	Scan specific configuration parameters, e.g. current coil and operation mode.
fid	Data	N. A.	Contains the acquired raw data for non-job based acquisition. See section D-1.3 for further information. This file is renamed to "ser" when the experiment is exported to <i>TOPSPIN</i> .

Table 1.3: Content of EXPNO directories

File Formats

Filename	Type	Paramter Group	Description
method	Parameter	MethodClass	Contains parameters of the PVM method used for the acquisition.
pulseprogram	Source Code	N. A.	Contains the source code of the pulse program used in the acquisition. This file may not be present as it is created when the acquisition is completed.
rawdata.jobN (N = Number)	Data	N. A.	Contains the acquired raw data for job based acquisition. For each defined acquisition job a file is created and filled with the raw data from this job.
spnamN (N = number)	Parameter	N. A.	These files contain the shape pulse definition used during acquisition. The files are created when the acquisition is completed.

Table 1.3: Content of EXPNO directories

The **PROCNO** directories may contain the following files:

Filename	Type		Description
2dseq	Data	N. A.	Image data file (see section D-1.4)
id	Parameter	DATASET_ID	Unique dataset identification.
d3proc	Parameter	D3	Description of the image data in 2dseq. Exists only for legacy datasets (e.g. <i>ParaVision 5.1</i>)
procs	Parameter	PROC	Contains additional parameters describing the image data (e.g. minimum, maximum values) used for processing in <i>TOPSPIN</i> .
methReco	Parameter	MethodReco-Group	Contains method specific input parameters for the reconstruction process.
reco	Parameter	RECO	Contains input and output parameters for the reconstruction process.
visu_pars	Parameter	Visu	Visu parameters for postprocessing, conversion, and display purposes (see section D-2.4.11).

Table 1.4: Content of PROCNO directories

Filename	Type		Description
roi	Parameter	ROI	Contains all Region of Interest definitions for this PROCNO.
isa	Parameter	ISA	Contains status information of the Image Sequence Analysis tool (see chapter O-2 and chapter D-5). It is required for PROCNOs of parameter images created by the ISA tool.
fun	Directory	N . A.	Contains files to describe the operation of the Functional Imaging Tool (see chapter O-3).

Table 1.4: Content of PROCNO directories

Parameter files

1.2

ParaVision writes parameter files in an enhanced JCAMP DX format (version 4.24). This section gives an introduction to the JCAMP DX format currently used by *ParaVision* in datasets. It must be mentioned that this format may change in future versions and that older datasets are not written exactly in the format described here. Information about the JCAMP DX format can be found at <http://www.jcamp.org/>.

The following description assumes that the reader is familiar with the JCAMP DX syntax which is described briefly in <http://www.jcamp.org/protocols/dxnmr01.pdf>.

The JCAMP DX format is not designed to describe image data. Image related parameters do not exist in JCAMP DX. Therefore, *ParaVision* uses only private parameters. Private parameters are defined with

File Formats

```
##$Parname = Value
```

e.g.

```
##$RG = 500
```

The value can be a simple value as defined in the JCAMP DX format (int, double), a string, an array value or a structure.

Arrays may contain preceding length indicators for every dimension of the array:

```
##$parname = ( dim1, dim2, ... )  
values
```

e. g. for ACQ_grad_matrix:

```
##$ACQ_grad_matrix = ( 1, 3, 3 )  
1 -0 -0 0 1 0 0 0 1
```

The different dimension lengths can be seen as limits of a loop for storing and reading arrays. The first length indicator defines the limits of the outer loop and the last length indicator the limits of the inner loop.

If the array lengths are fixed it is possible to omit the length indicator and to store the values directly after the parameter name. This is only supported for backward compatibility and should not be used anymore.

Strings are similar to arrays but the length indicator specifies the maximum length of the string. The string value has a preceding delimiter < and a succeeding delimiter >:

```
##$parname = ( max )  
<MyString>
```

e. g.

```
##$SPNAM0 = ( 16 )  
<gauss.exc>
```

A struct is written as a simple comma-separated sequence of elements:

```
##$parname = ( elem1, elem2, elem3 )
```

e. g.

```
##$InvPulse = (1.000, 1610.0, 180.0, 5.0, 100.00, 0.00, 100.00,  
LIB_INVERSION, <gauss.inv>, 1610.000000, 0.418172, 0.000000,  
0.102400,  
conventional)
```

File Formats

In this case arrays and strings do not have a preceding length indicator since they must all have static length.

The parameter files of *ParaVision* may store symbolic enum names or enum numbers. Normally the names are stored:

```
##$pname = <EnumValue>
```

e.g.

```
##$Method = <Bruker:FLASH>
```

If the enumeration has a special display name the enum name and display name are stored:

```
##$pname = (<EnumValue>, <EnumDisplayName>)
```

e.g.

```
##$$CONFIG_SCAN_operation_mode = (<$Bis,1,20131113...>,  
<[1H] TX Volume>)
```

Raw data files

1.3

The raw data is saved in the `fid` file in the `expno` directory (see [“Dataset paths” on page D-1-1](#)). This file may be renamed to `ser` if the experiment has been exported to *TopSpin*. The file contains the accumulated, but otherwise unprocessed data coming from the receiver, or, when the User Pipeline Filer is used, the output of the user filter. Raw data is not saved when the parameter **GO_data_save** is set to `No` (default setting is `Yes`). The data is saved in a binary format without a header with user-selected word types and block sizes.

Word type

Each data point is written as a pair of words: real part followed by imaginary part. The word type depends on the parameter **GO_raw_data_format** taking values:

- `GO_32_BIT_SGN_INT` for 32-bit signed integers (default). This is the only type supported by *TopSpin* processing.
- `GO_16_BIT_SGN_INT` for 16-bit signed integers. This setting may generate overflow errors in case of oversampling or accumulation and is not recommended.
- `GO_32_BIT_FLOAT` for floating point numbers.

File Formats

The byte order depends on the parameter **BYTORDA** taking values little (little-endian) or big (big-endian).

Block size

The data is written in blocks representing one scan (effect of a single **ADC_START** command in the pulse program). The blocks are written continuously when **GO_block_size** is set to `continuous` or, when this parameter is set to `Standard_KBlock_format`, the blocks are zero-filled to a multiple of 1kByte (1024 bytes). The latter setting is made by default and is required by the *TopSpin* processing.

Order of data storage

The data is saved in the order of acquisition, or, when the User Pipeline Filter is used, in the order defined by the filter output. The functional order of the data (relation of data points to echoes, slices, k-space lines etc.) is sequence specific and depends on several ACQP parameters such as **ACQ_dim**, **ACQ_size**, **NI**, **NR**, etc..

Image data files

1.4

All images reconstructed from one scan are saved in a single binary file `2dseq` in the `procno` directory (see ["Dataset paths" on page D-1-1](#)). The `2dseq` file contains pixel values without a header and without blockwise zero-filling. Pixel values are written sequentially, line by line, frame by frame, starting from the top left pixel of the first frame. The data type used depends on **RECO_word_type** (`_8BIT_UNSGN_INT`, `_16BIT_SGN_INT`, `_32BIT_SGN_INT`, `_32BIT_FLOAT`) while the byte order is given by **RECO_byte_order** (`littleEndian` or `bigEndian`). Unlike the raw data file, complex pixel values (reconstructed when **RECO_image_type** = `COMPLEX_IMAGE`) are not written in the interleaved manner; instead all real part frames are written first followed by all imaginary part frames.

